

Se detalla a continuación el modo en el que se maneja la lógica de la matriz para su posterior uso como salida en la matriz de luces HUB75.

Se consideró para este proyecto, a la matriz como un cuadrado de 32x32. Cada casillero se guarda como una estructura `Casillero_t`, en la cual hay 3 valores, siendo estos R, G, y B respectivamente.

```
typedef struct {  
    uint8_t r;  
    uint8_t g;  
    uint8_t b;  
} Casillero_t;
```

La matriz entonces, se guarda como una estructura `Matriz_t`, en la cual se tiene una grilla de datos `Casillero_t`.

```
typedef struct {  
    Casillero_t grilla[MATRIZ_FILAS][MATRIZ_COLUMNAS];  
} Matriz_t;
```

El TDA matriz incluye las siguientes funciones:

```
Matriz_t* matrizCrear(void);
```

Esta función permite crear una instancia de matriz, haciendo el manejo de memoria dinámica correspondiente. Devuelve un puntero a la matriz recientemente creada, y la inicializa con todos sus casilleros en negro. Si falla en crearla, devuelve NULL

```
void      matrizBorrar(Matriz_t* matrizEntrada);
```

Esta función elimina una instancia de la matriz, tomando un puntero a la matriz a destruir como argumento de entrada.

```
void      matrizSetCasillero(Matriz_t* matrizEntrada, uint8_t fila, uint8_t  
col, uint8_t r, uint8_t g, uint8_t b);
```

Esta función recibe como argumentos un puntero a la matriz a modificar, la fila y columna a modificar, y el color a poner en dicho casillero. Si se le da una ubicación fuera de los límites de la matriz, da un error a la consola de debugging y no hace nada más. De lo contrario, Pone el color recibido en el casillero indicado.

```
void matrizGetCasillero(Matriz_t* matrizEntrada, uint8_t fila, uint8_t col,  
Casillero_t* salida)
```

Esta función recibe como argumentos un puntero a la matriz, la fila y columna deseadas, y un puntero a `Casillero_t` en el que se devolverá el casillero deseado. Si los punteros no son NULL y la ubicación no está fuera de límites, hace que `Casillero_t* salida` esté dirigido hacia el casillero pedido.

```
void matrizGetColorCasillero(Matriz_t* matrizEntrada, uint8_t fila,
uint8_t col, uint8_t* entradaR, uint8_t* entradaG, uint8_t* entradaB);
```

Esta función recibe como argumentos un puntero a la matriz a modificar, la fila y columna deseadas, y 3 punteros `entradaR`, `entradaG`, y `entradaB` en los que se devuelven los valores RGB del casillero pedido.

```
void matrizLlenar(Matriz_t* matrizEntrada, uint8_t r, uint8_t g, uint8_t b);
```

Esta función recibe como argumentos un puntero a la matriz a modificar, y el color deseado. Se asume que `Matriz_t` apunta a una matriz válida. La función modifica el color de todos los casilleros en `matrizEntrada` y lo convierte en el color indicado.